

SQL Queries Based Interview Questions

1. Duplicate Customers

- **Question:** Find duplicate customers but keep the latest active record.
- **Approach:** `ROW_NUMBER()` partitioned by email, ordered by `updated_at DESC`.
- **Mistakes:** Using `DISTINCT`, ignoring ties.
- **Final Query:**

```
WITH ranked AS (  
    SELECT *, ROW_NUMBER() OVER (  
        PARTITION BY email ORDER BY updated_at DESC  
    ) AS rn  
    FROM customers  
)  
SELECT *  
FROM ranked  
WHERE rn = 1 AND status = 'active';
```

2. Top 3 Salaries per Department

- **Question:** Return top 3 salaries in each department, including ties.
- **Approach:** Use `DENSE_RANK()`.
- **Mistakes:** Using `ROW_NUMBER()` drops ties.
- **Final Query:**

```
WITH ranked AS (  
    SELECT *, DENSE_RANK() OVER (  
        PARTITION BY department_id ORDER BY salary DESC  
    ) AS dr  
    FROM employees  
)  
SELECT *  
FROM ranked  
WHERE dr <= 3;
```

3. 7-Day Login Streak

- **Question:** Identify users with at least 7 consecutive login days.
- **Approach:** Deduplicate dates, use `date - row_number` trick.

- **Mistakes:** Counting duplicate logins.
- **Final Query:**

```
WITH distinct_logins AS (  
    SELECT DISTINCT user_id, login_date FROM logins  
)  
marked AS (  
    SELECT user_id, login_date,  
           login_date - INTERVAL '1 day' *  
           ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) AS grp  
    FROM distinct_logins  
)  
SELECT user_id  
FROM marked  
GROUP BY user_id, grp  
HAVING COUNT(*) >= 7;
```

4. Missing Dates in Sales

- **Question:** Find missing dates in a sales table.
- **Approach:** Generate date series, anti-join with sales.
- **Mistakes:** Forgetting full calendar range.
- **Final Query:**

```
WITH RECURSIVE dates AS (  
    SELECT MIN(sale_date) AS dt  
    FROM sales  
    UNION ALL  
    SELECT dt + INTERVAL '1 day'  
    FROM dates  
    WHERE dt < (SELECT MAX(sale_date) FROM sales)  
)  
SELECT d.dt  
FROM dates d  
LEFT JOIN sales s ON s.sale_date = d.dt  
WHERE s.sale_date IS NULL;
```

5. Running Total with Reset

- **Question:** Compute running balance, reset when account closes.
- **Approach:** Use cumulative sum to create reset group.
- **Mistakes:** Single running sum only.
- **Final Query:**

```
WITH base AS (  
    SELECT account_id, txn_date, amount, status,  
           SUM(CASE WHEN status = 'closed' THEN 1 ELSE 0 END)  
           OVER (PARTITION BY account_id ORDER BY txn_date) AS reset_grp  
    FROM transactions  
)  
SELECT account_id, txn_date, amount,  
       SUM(amount) OVER (  
           PARTITION BY account_id, reset_grp ORDER BY txn_date  
       ) AS running_balance  
FROM base;
```

6. First Non-Null Phone

- **Question:** Find first non-null phone per user.
- **Approach:** Filter nulls, rank by timestamp.
- **Mistakes:** Using `MIN(phone_number)`.
- **Final Query:**

```
WITH filtered AS (  
    SELECT user_id, phone_number, updated_at,  
           ROW_NUMBER() OVER (  
               PARTITION BY user_id ORDER BY updated_at  
           ) AS rn  
    FROM user_contacts  
    WHERE phone_number IS NOT NULL  
)  
SELECT user_id, phone_number  
FROM filtered  
WHERE rn = 1;
```

7. Customers Without Orders

- **Question:** Find registered customers with no orders.
- **Approach:** `LEFT JOIN` + filter NULLs.
- **Mistakes:** Using `INNER JOIN`.
- **Final Query:**

```
SELECT c.customer_id, c.customer_name, c.email  
FROM customers c  
LEFT JOIN orders o ON c.customer_id = o.customer_id  
WHERE o.order_id IS NULL;
```

8. Products Above Average Sales

- **Question:** Products with sales > average.
- **Approach:** Aggregate per product, compare with subquery.
- **Mistakes:** Forgetting `GROUP BY`.
- **Final Query:**

```
SELECT p.product_id, p.product_name, SUM(s.amount) AS total_sales
FROM products p
JOIN sales s ON p.product_id = s.product_id
GROUP BY p.product_id, p.product_name
HAVING SUM(s.amount) > (
    SELECT AVG(total_sales)
    FROM (
        SELECT SUM(amount) AS total_sales
        FROM sales
        GROUP BY product_id
    ) t
);
```

9. Monthly Revenue Trend

- **Question:** Show monthly revenue + running total.
- **Approach:** Aggregate by month, window sum.
- **Mistakes:** Ordering by string month.
- **Final Query:**

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
       SUM(amount) AS monthly_revenue,
       SUM(SUM(amount)) OVER (
           ORDER BY DATE_FORMAT(order_date, '%Y-%m')
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_revenue
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

10. Top 3 Customers per Category

- **Question:** Top 3 customers by spending per category.
- **Approach:** `ROW_NUMBER()` partitioned by category.
- **Mistakes:** Using `RANK()`/`DENSE_RANK()`.

- **Final Query:**

```
WITH ranked AS (
  SELECT c.customer_id, c.customer_name,
         p.category_id, cat.category_name,
         SUM(o.amount) AS total_spent,
         ROW_NUMBER() OVER (
           PARTITION BY p.category_id ORDER BY SUM(o.amount) DESC
         ) AS rn
  FROM customers c
  JOIN orders o ON c.customer_id = o.customer_id
  JOIN products p ON o.product_id = p.product_id
  JOIN categories cat ON p.category_id = cat.category_id
  GROUP BY c.customer_id, c.customer_name,
           p.category_id, cat.category_name
)
SELECT customer_id, customer_name, category_id, category_name, total_spent
FROM ranked
WHERE rn <= 3
ORDER BY category_id, total_spent DESC;
```

11. Employees Paid More Than Manager

- **Question:** Employees earning more than manager.
- **Approach:** Self join employees table.
- **Mistakes:** Wrong join, ignoring null managers.
- **Final Query:**

```
SELECT e.employee_id, e.employee_name, e.salary,
       m.employee_name AS manager_name, m.salary AS manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;
```

12. Consecutive Login Days

- **Question:** Find streak start, end, total days.
- **Approach:** `date - row_number()` trick.
- **Mistakes:** Counting duplicates.
- **Final Query:**

```

WITH ordered AS (
    SELECT user_id, CAST(login_date AS DATE) AS login_date,
           ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) AS rn
    FROM logins
),
grouped AS (
    SELECT user_id, login_date,
           DATE_SUB(login_date, INTERVAL rn DAY) AS grp_key
    FROM ordered
)
SELECT user_id,
       MIN(login_date) AS start_date,
       MAX(login_date) AS end_date,
       COUNT(*) AS total_days
FROM grouped
GROUP BY user_id, grp_key
ORDER BY user_id, start_date;

```

13. Cumulative Product Sales

- **Question:** Show cumulative monthly sales per product.
- **Approach:** Aggregate monthly, window sum.
- **Mistakes:** Not ordering months.
- **Final Query:**

```

WITH monthly_sales AS (
    SELECT product_id,
           DATE_FORMAT(sale_date, '%Y-%m') AS ym,
           SUM(amount) AS monthly_sales
    FROM sales
    GROUP BY product_id, DATE_FORMAT(sale_date, '%Y-%m')
)
SELECT product_id,
       ym AS month,
       monthly_sales,
       SUM(monthly_sales) OVER (
           PARTITION BY product_id ORDER BY ym
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS cumulative_sales
FROM monthly_sales
ORDER BY product_id, ym;

```

14. Departments with >5 Employees

- **Question:** Find departments with more than 5 employees.

- **Approach:** Group + HAVING.
- **Mistakes:** Using `WHERE`.
- **Final Query:**

```
SELECT department_id, department_name, COUNT(*) AS emp_count
FROM employees
GROUP BY department_id, department_name
HAVING COUNT(*) > 5
ORDER BY emp_count DESC;
```

15. Second Highest Salary

- **Question:** Find second highest salary.
- **Approach:** Use `DISTINCT` + `LIMIT/OFFSET`.
- **Mistakes:** Using `MAX()` twice.
- **Final Query:**

```
SELECT salary
FROM (
    SELECT DISTINCT salary
    FROM employees
    ORDER BY salary DESC
) t
LIMIT 1 OFFSET 1;
```

16. Customers Spending More in 2024 vs 2023

- **Question:** Customers with higher spend in 2024.
- **Approach:** Aggregate yearly spend, self join.
- **Mistakes:** Wrong year join, missing customers.
- **Final Query:**

```
WITH yearly_spend AS (
    SELECT customer_id, YEAR(order_date) AS yr,
           SUM(amount) AS total_spent
    FROM orders
    WHERE YEAR(order_date) IN (2023, 2024)
    GROUP BY customer_id, YEAR
```